How significant are kills in the NBA?

Shashwat Mishra

2026-02-02

Kills in the NBA: Testing Sean Miller's theory of kills & win percentage

There really are a lot of new things you learn in college outside of class. Sometimes, it can happen at a basketball game. After the departure of Texas phenom Tre Johnson to the NBA, and subsequently, coach Rodney Terry (after a miraculous entry into the tournament and proceeding 1st round exit), I was excited to see Sean Miller take over.

After going to the Longhorns' season opener, the pregame video was about a stat I'd never heard before: kills. A kill is simple, 3 defensive stops in a row, be it by missed shot + defensive rebound, offensive foul, steal, etc. Miller proclaimed that reaching the area around 7+ kills brings win probability of the game up to 90%.

This project just look at how kills translate to the NBA, a league dominated by star-studded offenses.

Part 1: Testing Scope: Using a smaller sample (Dallas Mavericks 2024-2025)

1. Load NBA play-by-play data for the 2024-25 season

```
season24_25 <- load_nba_pbp(
  seasons = 2025
)
```

2. Look at only Mavs games

```
mavs_2025 <- season24_25 %>%
  filter(home_team_abbrev == "DAL" | away_team_abbrev == "DAL")
```

3. Analyzing Defensive Stops by text data

```
mavs_2025 <- mavs_2025 %>%
  arrange(game_id, period, desc(clock_seconds)) %>%
  mutate(
    mavs_on_defense = case_when(
      home_team_abbrev == "DAL" & team_id == away_team_id ~ TRUE,
```

```r
      away_team_abbrev == "DAL" & team_id == home_team_id ~ TRUE,
      TRUE ~ FALSE
    ),

    opp_turnover = mavs_on_defense &
      grepl(
      "Turnover|Offensive Foul|Violation",
      type_text,
      ignore.case = TRUE
    ),

    opp_missed_shot = mavs_on_defense &
      grepl("misses", text),

    mavs_def_reb = !mavs_on_defense &
      type_text == "Defensive Rebound"
  )

#Organize by game in order
mavs_2025 <- mavs_2025 %>%
  arrange(
    game_id,
    period_number,
    desc(clock_minutes),
    desc(clock_seconds)
  )

mavs_2025 <- mavs_2025 %>%
  mutate(
    opp_scored = mavs_on_defense & scoring_play
  )
```

4. Defensive Stops

```r
mavs_2025 <- mavs_2025 %>%
  mutate(
    defensive_stop = case_when(
      opp_turnover ~ 1,
      opp_missed_shot & lead(mavs_def_reb, default = FALSE) ~ 1,
      TRUE ~ 0
    )
  )
```

5. Making a possession index

```r
# Create possession index
mavs_2025 <- mavs_2025 %>%
  arrange(
    game_id,
    period_number,
```

```r
    desc(clock_minutes),
    desc(clock_seconds)
  ) %>%
  mutate(
    possession_change = case_when(
      opp_turnover ~ 1,
      mavs_def_reb ~ 1,
      scoring_play ~ 1,
      TRUE ~ 0
    )
  ) %>%
  group_by(game_id) %>%
  mutate(
    possession = cumsum(possession_change)
  ) %>%
  ungroup()
```

6. Kills logic + merging onto original data

```r
stops_by_possession <- mavs_2025 %>%
  group_by(game_id, possession) %>%
  summarise(
    stop = as.integer(any(defensive_stop == 1)),
    .groups = "drop"
  )

kills_by_possession <- stops_by_possession %>%
  arrange(game_id, possession) %>%
  group_by(game_id) %>%
  mutate(
    stop_streak = ifelse(
      stop == 1,
      ave(stop, cumsum(stop == 0), FUN = cumsum),
      0
    ),
    kill = ifelse(stop_streak %% 3 == 0 & stop_streak > 0, 1, 0)
  ) %>%
  ungroup()


kills_per_game <- kills_by_possession %>%
  group_by(game_id) %>%
  summarise(
    kills = sum(kill),
    .groups = "drop"
  )
```

```r
# Sanity Check
kills_by_possession %>%
  filter(game_id == 401703376) %>%
  select(possession, stop, stop_streak, kill)
```

```
## # A tibble: 174 x 4
##    possession  stop stop_streak  kill
##         <dbl> <int>       <dbl> <dbl>
##  1          0     0           0     0
##  2          1     0           0     0
##  3          2     0           0     0
##  4          3     0           0     0
##  5          4     0           0     0
##  6          5     1           1     0
##  7          6     0           0     0
##  8          7     0           0     0
##  9          8     0           0     0
## 10          9     1           1     0
## # i 164 more rows
```

7. Merging with Mavs wins

```r
mavs_games <- load_nba_team_box(seasons = 2025) %>%
  filter(team_abbreviation == "DAL")

# Merge kills with results
mavs_kills_wins <- kills_per_game %>%
  left_join(mavs_games, by = "game_id")
```

```r
kills_per_game <- kills_by_possession %>%
  group_by(game_id) %>%
  summarise(
    kills = sum(kill),
    .groups = "drop"
  )

nrow(kills_per_game)
```

```
## [1] 75
```

```r
mavs_kills_games <- mavs_games %>%
  left_join(
    kills_per_game,
    by = "game_id"
  ) %>%
  mutate(
    kills = replace_na(kills, 0)
  )
#count total number of kills
mavs_kills_games %>%
  summarise(total_kills = sum(kills))
```

```
## # A tibble: 1 x 1
##   total_kills
##         <dbl>
## 1          93
```

```r
mavs_kills_games <- mavs_kills_games %>%
  mutate(win_num = as.integer(team_winner))
```

8. Logistic Regression Model

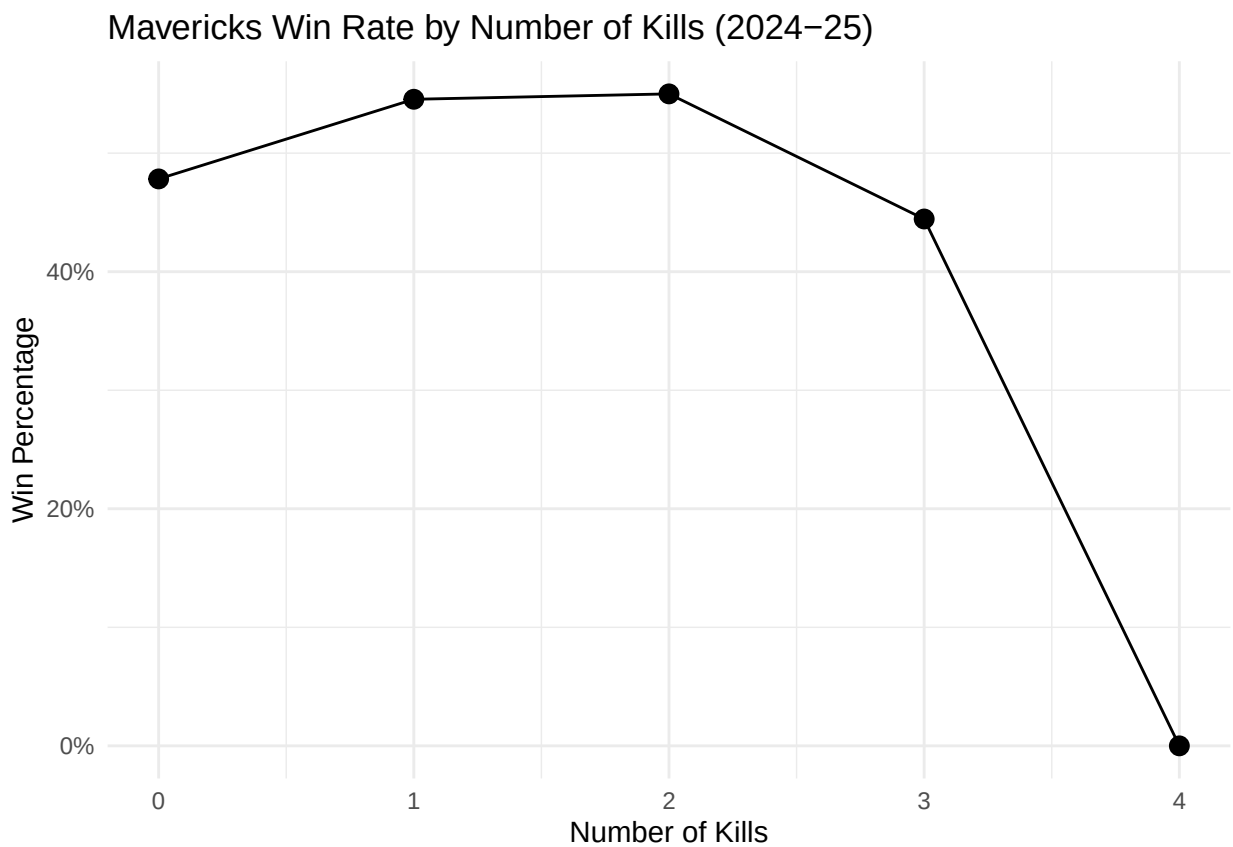
```r
kills_model <- glm(
  team_winner ~ kills,
  data = mavs_kills_games,
  family = binomial
)

summary(kills_model)
```

```
##
## Call:
## glm(formula = team_winner ~ kills, family = binomial, data = mavs_kills_games)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.09311    0.35646   0.261    0.794
## kills       -0.05357    0.21881  -0.245    0.807
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 103.96  on 74  degrees of freedom
## Residual deviance: 103.90  on 73  degrees of freedom
## AIC: 107.9
##
## Number of Fisher Scoring iterations: 3
```

9. Graphing the relationship between kills and win percentage

```r
mavs_kill_summary <- mavs_kills_games %>%
  group_by(kills) %>%
  summarise(


    win_rate = mean(win_num),
    n = n(),
    .groups = "drop"
  )

ggplot(mavs_kill_summary, aes(x = kills, y = win_rate)) +
  geom_point(size = 3) +
  geom_line() +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    title = "Mavericks Win Rate by Number of Kills (2024-25)",
    x = "Number of Kills",
```

```
    y = "Win Percentage"
  ) +
  theme_minimal()
```



- home & away team kills, does one team has a lot of kills, does the other team have as many or significantly less.
- find correlation with other defensive metrics (BPG, SPG, etc)
- model with DRTG instead of kills to see difference (compare and see which is better)
- look at the distribution of kills per game, see if there are outliers, and how many games have 7+ kills (the threshold Miller mentioned) and how many of those were wins.

2. Moving into the NBA: Expanding the scope to all teams and seasons

```
# Kills in the NBA: Full League Analysis (2024-25 + Multi-Season)
library(dplyr)
library(ggplot2)
library(ggthemes)
library(hoopR)
library(tidyr)
```

```r
library(lme4)
library(scales)
library(purrr)
```

```r
# HELPER FUNCTION: Compute kills for one team in one game
ALL_STAR_TEAMS <- c("CHK", "SHQ", "KEN", "CAN")


compute_kills_for_team <- function(pbp_data, team_abbrev) {
  team_games <- pbp_data %>%
    filter(home_team_abbrev == team_abbrev | away_team_abbrev == team_abbrev)

  team_games <- team_games %>%
    arrange(game_id, period_number, desc(clock_minutes), desc(clock_seconds)) %>%
    mutate(
      team_on_defense = case_when(
        home_team_abbrev == team_abbrev & team_id == away_team_id ~ TRUE,
        away_team_abbrev == team_abbrev & team_id == home_team_id ~ TRUE,
        TRUE ~ FALSE
      ),
      opp_turnover = team_on_defense &
        grepl("Turnover|Offensive Foul|Violation", type_text, ignore.case = TRUE),
      opp_missed_shot = team_on_defense & grepl("misses", text),
      team_def_reb = !team_on_defense & type_text == "Defensive Rebound"
    ) %>%
    mutate(
      defensive_stop = case_when(
        opp_turnover ~ 1,
        opp_missed_shot & lead(team_def_reb, default = FALSE) ~ 1,
        TRUE ~ 0
      ),
      possession_change = case_when(
        opp_turnover ~ 1,
        team_def_reb ~ 1,
        scoring_play ~ 1,
        TRUE ~ 0
      )
    ) %>%
    group_by(game_id) %>%
    mutate(possession = cumsum(possession_change)) %>%
    ungroup()

  stops_by_possession <- team_games %>%
    group_by(game_id, possession) %>%
    summarise(stop = as.integer(any(defensive_stop == 1)), .groups = "drop")

  kills_by_possession <- stops_by_possession %>%
    arrange(game_id, possession) %>%
    group_by(game_id) %>%
    mutate(
      stop_streak = ifelse(
        stop == 1,
        ave(stop, cumsum(stop == 0), FUN = cumsum),
```

```r
        0
      ),
      kill = ifelse(stop_streak %% 3 == 0 & stop_streak > 0, 1, 0)
    ) %>%
    ungroup()

  kills_by_possession %>%
    group_by(game_id) %>%
    summarise(kills = sum(kill), team_abbrev = team_abbrev, .groups = "drop")
}
```

1. League Analysis for the 2024-2025 NBA Season

```r
# PART 1: 2024-25 Full League Analysis

season_pbp <- load_nba_pbp(seasons = 2025)

all_teams <- unique(c(season_pbp$home_team_abbrev, season_pbp$away_team_abbrev))
all_teams <- all_teams[!is.na(all_teams) & !all_teams %in% ALL_STAR_TEAMS]

# Compute kills for all teams (takes a few minutes)
all_kills <- map_dfr(all_teams, function(tm) {
  message("Processing: ", tm)
  compute_kills_for_team(season_pbp, tm)
})
```

```
## Processing: OKC

## Processing: IND

## Processing: NY

## Processing: MIN

## Processing: DEN

## Processing: BOS

## Processing: CLE

## Processing: GS

## Processing: HOU

## Processing: DET

## Processing: LAC

## Processing: LAL
```

```
## Processing: MIA

## Processing: ORL

## Processing: MIL

## Processing: MEM

## Processing: CHI

## Processing: SAC

## Processing: PHI

## Processing: NO

## Processing: DAL

## Processing: UTAH

## Processing: PHX

## Processing: POR

## Processing: CHA

## Processing: BKN

## Processing: ATL

## Processing: SA

## Processing: WSH

## Processing: TOR
```

```r
# Load team box scores for all teams
team_box <- load_nba_team_box(seasons = 2025)

# Join kills with results
kills_wins <- team_box %>%
  left_join(all_kills, by = c("game_id", "team_abbreviation" = "team_abbrev")) %>%
  mutate(
    kills = replace_na(kills, 0),
    win_num = as.integer(team_winner)
  )

# (kills allowed per game)
# Merge home and away kills to get both sides
```

```r
home_kills <- all_kills %>%
  rename(home_team_abbrev = team_abbrev, home_kills = kills)
away_kills <- all_kills %>%
  rename(away_team_abbrev = team_abbrev, away_kills = kills)

game_kills <- season_pbp %>%
  distinct(game_id, home_team_abbrev, away_team_abbrev) %>%
  left_join(home_kills, by = c("game_id", "home_team_abbrev")) %>%
  left_join(away_kills, by = c("game_id", "away_team_abbrev"))

kills_wins <- kills_wins %>%
  left_join(
    game_kills %>% select(game_id, home_team_abbrev, away_team_abbrev, home_kills, away_kills),
    by = "game_id"
  ) %>%
  mutate(
    opp_kills = case_when(
      team_abbreviation == home_team_abbrev ~ away_kills,
      team_abbreviation == away_team_abbrev ~ home_kills,
      TRUE ~ NA_real_
    ),
    kills_differential = kills - opp_kills
  )
```

2. Logistic Regression Model (Regressing Wins on # Of Kills)

```r
# PART 2: Logistic Regression (full league)

# Simple logistic: kills ~ wins
kills_model_full <- glm(
  team_winner ~ kills,
  data = kills_wins,
  family = binomial
)
summary(kills_model_full)
```

```
##
## Call:
## glm(formula = team_winner ~ kills, family = binomial, data = kills_wins)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.21303    0.06390  -3.334 0.000856 ***
## kills        0.16073    0.03722   4.318 1.57e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 3352.1  on 2417  degrees of freedom
## Residual deviance: 3333.1  on 2416  degrees of freedom
```

```
## AIC: 3337.1
##
## Number of Fisher Scoring iterations: 4
```

```r
# With kills differential (stronger signal)
kills_diff_model <- glm(
  team_winner ~ kills_differential,
  data = kills_wins,
  family = binomial
)
summary(kills_diff_model)
```

```
##
## Call:
## glm(formula = team_winner ~ kills_differential, family = binomial,
##     data = kills_wins)
##
## Coefficients:
##                      Estimate Std. Error z value Pr(>|z|)
## (Intercept)        -1.070e-18  4.113e-02   0.000        1
## kills_differential  2.042e-01  3.001e-02   6.802 1.03e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 3343.7  on 2411  degrees of freedom
## Residual deviance: 3295.9  on 2410  degrees of freedom
##   (6 observations deleted due to missingness)
## AIC: 3299.9
##
## Number of Fisher Scoring iterations: 4
```

```r
# Mixed effects model (accounts for team-level variation)
kills_mixed <- glmer(
  team_winner ~ kills + (1 | team_abbreviation),
  data = kills_wins,
  family = binomial
)
summary(kills_mixed)
```

```
## Generalized linear mixed model fit by maximum likelihood (Laplace
##   Approximation) [glmerMod]
##  Family: binomial  ( logit )
## Formula: team_winner ~ kills + (1 | team_abbreviation)
##    Data: kills_wins
##
##      AIC      BIC   logLik deviance df.resid
##   3200.7   3218.1  -1597.3   3194.7     2415
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -2.3989 -0.9254 -0.0303  0.9118  2.0143
```

```
##
## Random effects:
##  Groups            Name        Variance Std.Dev.
##  team_abbreviation (Intercept) 0.3773   0.6142
## Number of obs: 2418, groups:  team_abbreviation, 34
##
## Fixed effects:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.26167    0.12951  -2.021   0.0433 *
## kills        0.16987    0.03902   4.353 1.34e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##       (Intr)
## kills -0.393
```

3: Looking at Miller's 7+ Kill Threshold

```r
# PART 3: Miller's 7+ Kill Threshold Test

kills_wins <- kills_wins %>%
  mutate(seven_plus_kills = kills >= 7)

threshold_summary <- kills_wins %>%
  group_by(seven_plus_kills) %>%
  summarise(
    win_rate = mean(win_num, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )

print(threshold_summary)
```

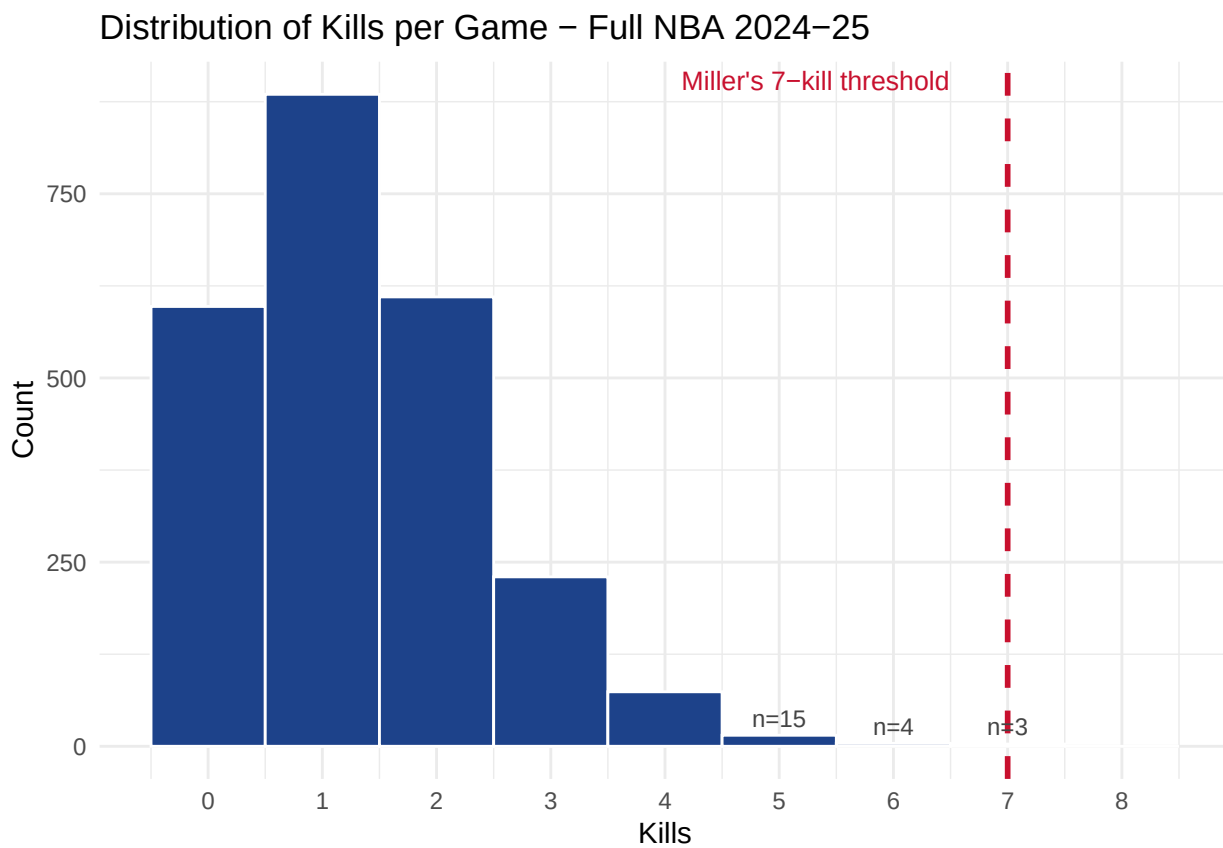
```
## # A tibble: 2 x 3
##   seven_plus_kills win_rate     n
##   <lgl>               <dbl> <int>
## 1 FALSE               0.499  2415
## 2 TRUE                1         3
```

```r
# Win rate across kill buckets
kill_bucket_summary <- kills_wins %>%
  mutate(kill_bucket = cut(kills, breaks = c(-Inf, 2, 4, 6, 8, 10, Inf),
                           labels = c("0-2", "3-4", "5-6", "7-8", "9-10", "11+"))) %>%
  group_by(kill_bucket) %>%
  summarise(
    win_rate = mean(win_num, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
```

```r
seven_kills <- kills_wins %>%
  filter(seven_plus_kills == TRUE)
```

4. Distribution of Kills per Game + Win Rate by Kills (Full League)

```r
# PART 4: Distribution of Kills per Game

kill_counts <- kills_wins %>%
  count(kills) %>%
  filter(kills >= 5)  # only label the sparse ones

p_dist <- ggplot(kills_wins, aes(x = kills)) +
  geom_histogram(binwidth = 1, fill = "#1D428A", color = "white") +
  geom_vline(xintercept = 7, color = "#C8102E", linetype = "dashed", linewidth = 1) +
  geom_text(data = kill_counts, aes(x = kills, y = n, label = paste0("n=", n)),
            vjust = -0.5, size = 3, color = "#444") +
  annotate("text", x = 6.5, y = Inf, label = "Miller's 7-kill threshold",
           hjust = 1, vjust = 1.5, color = "#C8102E", size = 3.5) +
  scale_x_continuous(breaks = 0:8, limits = c(-0.5, 8.5)) +
  labs(
    title = "Distribution of Kills per Game - Full NBA 2024-25",
    x = "Kills", y = "Count"
  ) +
  theme_minimal()

p_dist
```




```
lambda <- mean(kills_wins$kills, na.rm = TRUE)  # estimated rate parameter

# P(X <= 2) under Poisson
ppois(2, lambda = lambda)
```

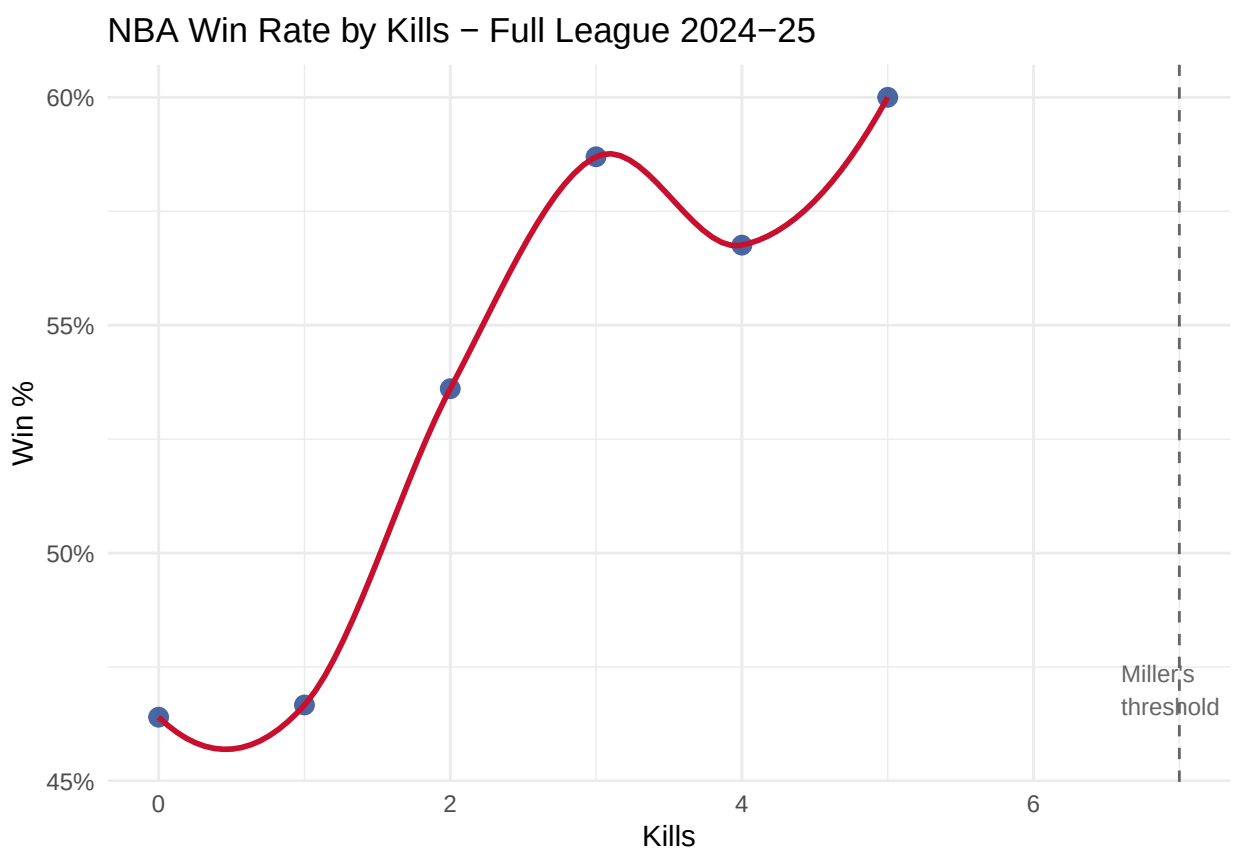
```
## [1] 0.8506266
```

5. Win Rate by Kills

```
# PART 5: Win Rate by Kills - Full League
library(scales)
league_kill_summary <- kills_wins %>%
  group_by(kills) %>%
  summarise(
    win_rate = mean(win_num, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  ) %>%
  filter(n >= 10)  # filter sparse kill counts

p_winrate <- ggplot(league_kill_summary, aes(x = kills, y = win_rate)) +
  geom_point(color = "#1D428A", alpha = 0.8, size = 3) +
  geom_smooth(aes(weight = n), method = "loess", se = TRUE,
```

```r
              color = "#C8102E", fill = "#C8102E", alpha = 0.15) +
  geom_vline(xintercept = 7, linetype = "dashed", color = "gray40") +
  annotate("text", x = 6.6, y = 0.47, label = "Miller's\nthreshold",
           color = "gray40", size = 3, hjust = 0) +
  scale_y_continuous(labels = percent_format()) +
  labs(
    title = "NBA Win Rate by Kills - Full League 2024-25",
    x = "Kills", y = "Win %"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
p_winrate
```

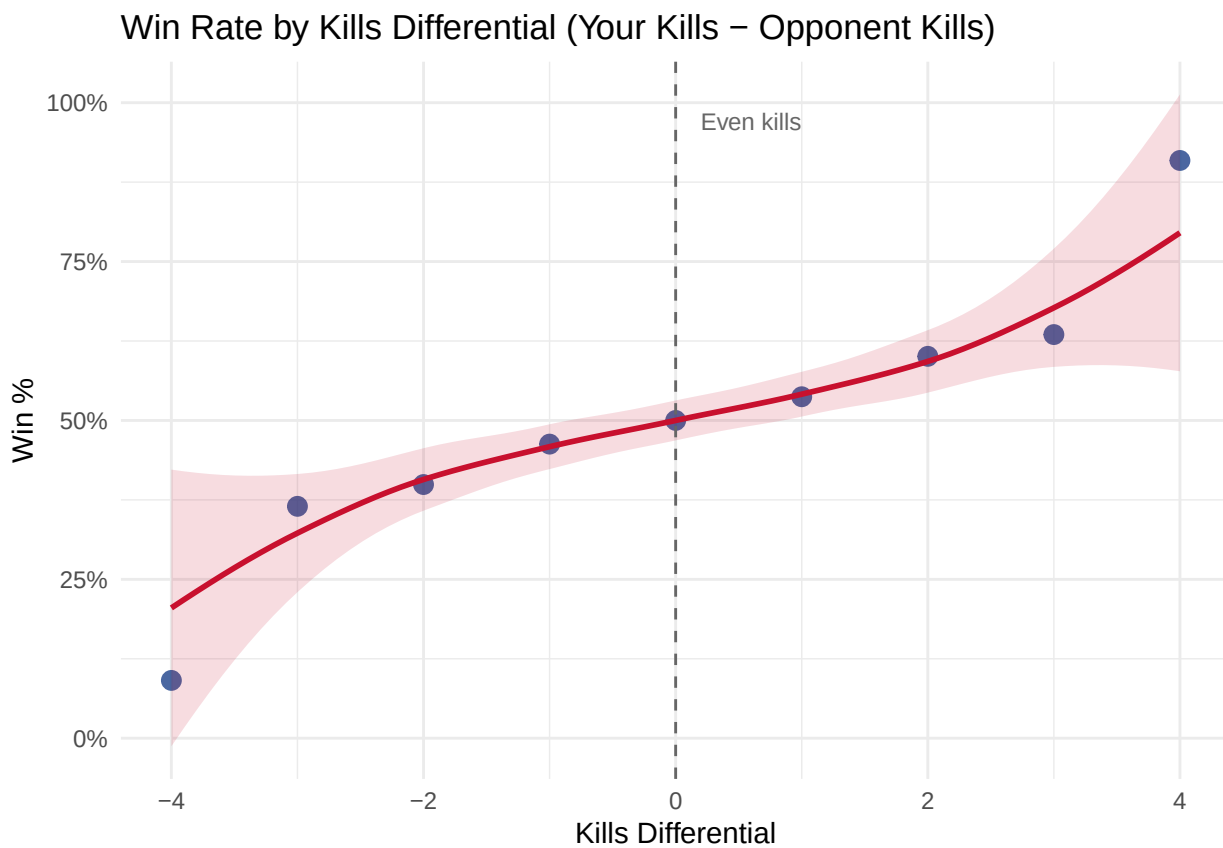
```
## `geom_smooth()` using formula = 'y ~ x'

## Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
## : Chernobyl! trL>n 6
## Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
## : Chernobyl! trL>n 6

## Warning in sqrt(sum.squares/one.delta): NaNs produced

## Warning in stats::qt(level/2 + 0.5, pred$df): NaNs produced

## Warning in max(ids, na.rm = TRUE): no non-missing arguments to max; returning
## -Inf
```



6. Kills Differential vs. Win Rate

```r
# PART 6: Kills Differential vs Win Rate

diff_summary <- kills_wins %>%
  group_by(kills_differential) %>%
  summarise(
    win_rate = mean(win_num, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  ) %>%
  filter(n >= 10)

p_diff <- ggplot(diff_summary, aes(x = kills_differential, y = win_rate)) +
  geom_point(color = "#1D428A", alpha = 0.8, size = 3) +
  geom_smooth(aes(weight = n), method = "loess", se = TRUE,
              color = "#C8102E", fill = "#C8102E", alpha = 0.15) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray40") +
  annotate("text", x = 0.2, y = 0.97, label = "Even kills",
           color = "gray40", size = 3, hjust = 0) +
  scale_y_continuous(labels = percent_format()) +
  labs(
    title = "Win Rate by Kills Differential (Your Kills - Opponent Kills)",
    x = "Kills Differential", y = "Win %"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
p_diff
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



7. Kills Per Team (Total + Averages)

```
# PART 7: Team-Level Kills Leaderboard (season averages)

team_kill_avgs <- kills_wins %>%
  group_by(team_abbreviation) %>%
  summarise(
    avg_kills = mean(kills, na.rm = TRUE),
    avg_opp_kills = mean(opp_kills, na.rm = TRUE),
    avg_diff = mean(kills_differential, na.rm = TRUE),
    win_pct = mean(win_num, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(avg_kills))

print(team_kill_avgs)
```

```
## # A tibble: 34 x 5
##    team_abbreviation avg_kills avg_opp_kills avg_diff win_pct
##    <chr>                 <dbl>         <dbl>    <dbl>   <dbl>
##  1 MIN                    1.59          1.28   0.311    0.578
##  2 CHA                    1.58          1.71  -0.137    0.247
##  3 MIA                    1.49          1.08   0.418    0.430
```

```
##  4 IND                    1.49          1.11   0.378    0.602
##  5 MIL                    1.48          1.35   0.125    0.575
##  6 PHX                    1.45          1.41   0.0400   0.467
##  7 CHI                    1.44          1.47  -0.0385   0.449
##  8 OKC                    1.43          1.40   0.0303   0.788
##  9 WSH                    1.41          1.75  -0.333    0.2
## 10 HOU                    1.41          1.10   0.309    0.630
## # i 24 more rows
```

```r
# Scatter: avg kills vs win pct by team

p_team <- team_kill_avgs %>%
  filter(!team_abbreviation %in% ALL_STAR_TEAMS) %>%
  ggplot(aes(x = avg_diff, y = win_pct)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray60") +
  geom_hline(yintercept = 0.5, linetype = "dashed", color = "gray60") +
  geom_nba_logos(aes(team_abbr = team_abbreviation), width = 0.055, alpha = 0.9) +
  scale_y_continuous(labels = percent_format()) +
  labs(title = "Kills Differential vs Win % by Team - 2024-25 NBA",
       subtitle = "Teams in top-right quadrant win more AND out-kill opponents",
       x = "Avg Kills Differential (Kills - Opp Kills)", y = "Win %") +
  theme_minimal(base_size = 12)
```

8. Multi-Season Analysis (2020-2025)

```r
# PART 8: Multi-Season Analysis (2020-2025)

multi_team_box <- load_nba_team_box(seasons = 2020:2025)

# Compute kills season by season, tagging each with the season year
seasons <- 2020:2025

multi_season_kills <- map_dfr(seasons, function(s) {
  message("Loading PBP for season: ", s)
  pbp <- load_nba_pbp(seasons = s)  # load one season at a time to avoid memory issues

  teams <- unique(c(pbp$home_team_abbrev, pbp$away_team_abbrev))
  teams <- teams[!is.na(teams) & !teams %in% ALL_STAR_TEAMS]

  map_dfr(teams, function(tm) {
    compute_kills_for_team(pbp, tm) %>% mutate(season = s)
  })
})
```

```
## Loading PBP for season: 2020

## Loading PBP for season: 2021

## Loading PBP for season: 2022

## Loading PBP for season: 2023
```

```
## Loading PBP for season: 2024

## Loading PBP for season: 2025
```

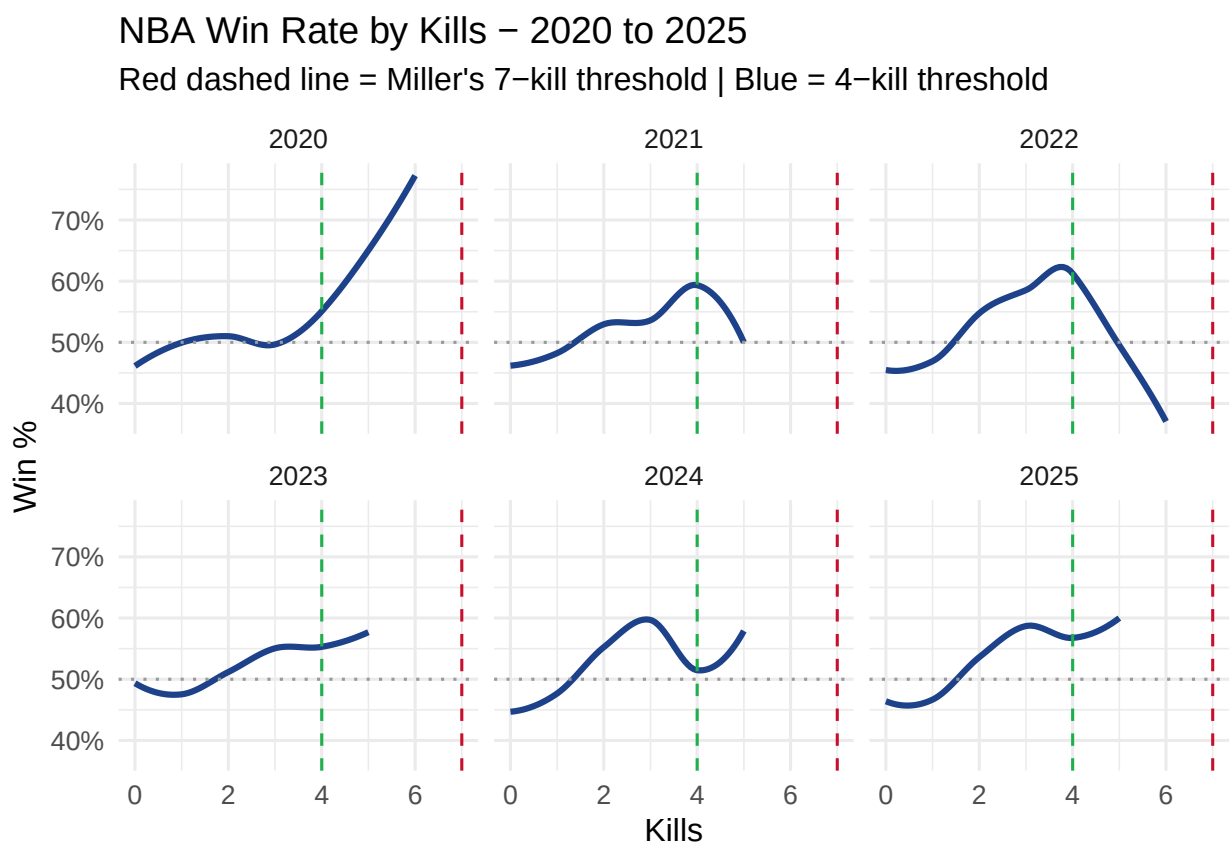
```r
# Join on game_id only - season is just for faceting later
multi_kills_wins <- multi_team_box %>%
  left_join(
    multi_season_kills %>% select(game_id, team_abbrev, kills, season),
    by = c("game_id", "team_abbreviation" = "team_abbrev")
  ) %>%
  mutate(
    kills   = replace_na(kills, 0),
    win_num = as.integer(team_winner)
  )

# Multi-season logistic regression
multi_model <- glm(team_winner ~ kills, data = multi_kills_wins, family = binomial)
summary(multi_model)
```

```
##
## Call:
## glm(formula = team_winner ~ kills, family = binomial, data = multi_kills_wins)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.16679    0.02572  -6.485 8.85e-11 ***
## kills        0.12077    0.01436   8.413  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 20628  on 14879  degrees of freedom
## Residual deviance: 20557  on 14878  degrees of freedom
## AIC: 20561
##
## Number of Fisher Scoring iterations: 3
```

```r
# Win rate by kills, faceted by season
multi_season_summary <- multi_kills_wins %>%
  group_by(season.x, kills) %>%
  summarise(win_rate = mean(win_num, na.rm = TRUE), n = n(), .groups = "drop") %>%
  filter(n >= 5)

p_multi <- ggplot(multi_season_summary, aes(x = kills, y = win_rate)) +
  geom_smooth(aes(weight = n), method = "loess", se = FALSE, color = "#1D428A") +
  geom_vline(xintercept = 7, linetype = "dashed", color = "#C8102E") +
  geom_vline(xintercept = 4, linetype = "dashed", color = "#1EB04C") +
  geom_hline(yintercept = 0.5, linetype = "dotted", color = "gray60") +
  scale_y_continuous(labels = percent_format()) +
  facet_wrap(~season.x) +
  labs(
    title = "NBA Win Rate by Kills - 2020 to 2025",
```

```
    subtitle = "Red dashed line = Miller's 7-kill threshold | Blue = 4-kill threshold",
    x = "Kills", y = "Win %"
  ) +
  theme_minimal(base_size = 12)

print(p_multi)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



9. Kills & DRTG

```
# PART 9: Kills vs DRTG (Defensive Rating) Comparison

if ("defensive_rating" %in% colnames(kills_wins)) {
  drtg_model <- glm(
    team_winner ~ defensive_rating,
    data = kills_wins,
    family = binomial
  )

  kills_vs_drtg <- kills_wins %>%
    filter(!is.na(kills), !is.na(defensive_rating)) %>%
```

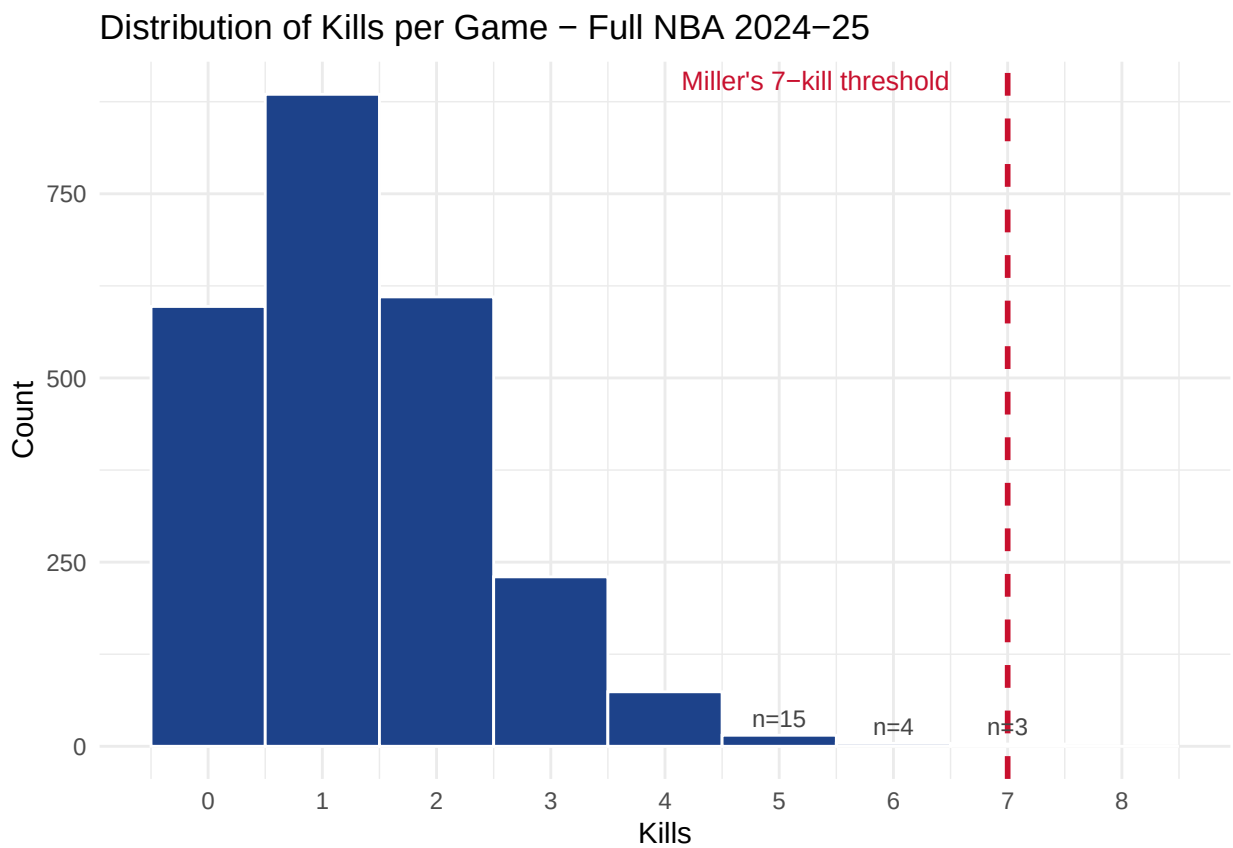
```r
    summarise(
      kills_AIC = AIC(kills_model_full),
      drtg_AIC  = AIC(drtg_model)
    )

  print(kills_vs_drtg)  # Lower AIC = better fit
}
```

10. Win % by kill bucket

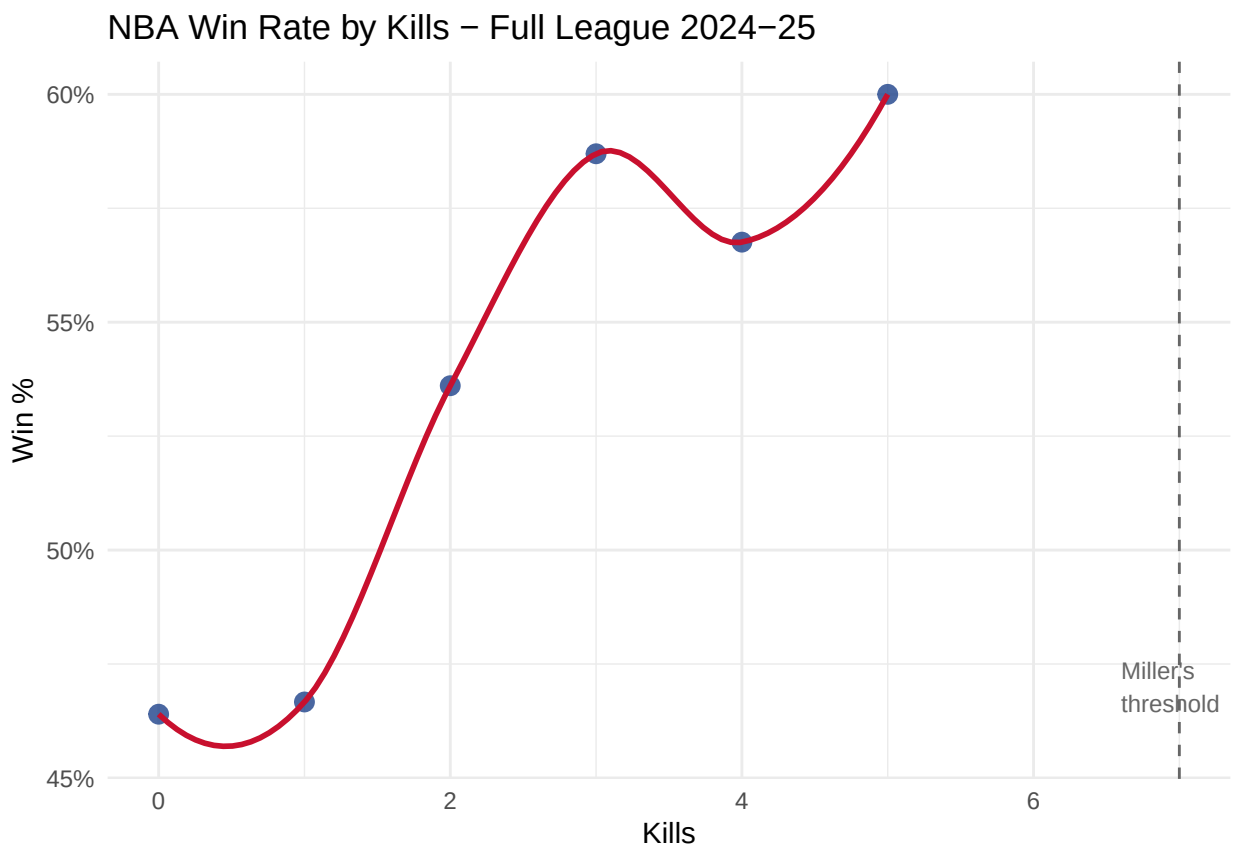
```r
p_bucket <- ggplot(kill_bucket_summary, aes(x = kill_bucket, y = win_rate, fill = kill_bucket)) +
  geom_col() +
  geom_text(aes(label = paste0(round(win_rate * 100, 1), "%\n(n=", n, ")")),
            vjust = -0.3, size = 3) +
  scale_y_continuous(labels = percent_format(), limits = c(0, 0.75)) +
  scale_fill_brewer(palette = "Blues", guide = "none") +
  labs(title = "Win Rate by Kill Bucket - Full NBA 2024-25",
       x = "Kill Range", y = "Win %") +
  theme_minimal()
```

11. Print all Plots

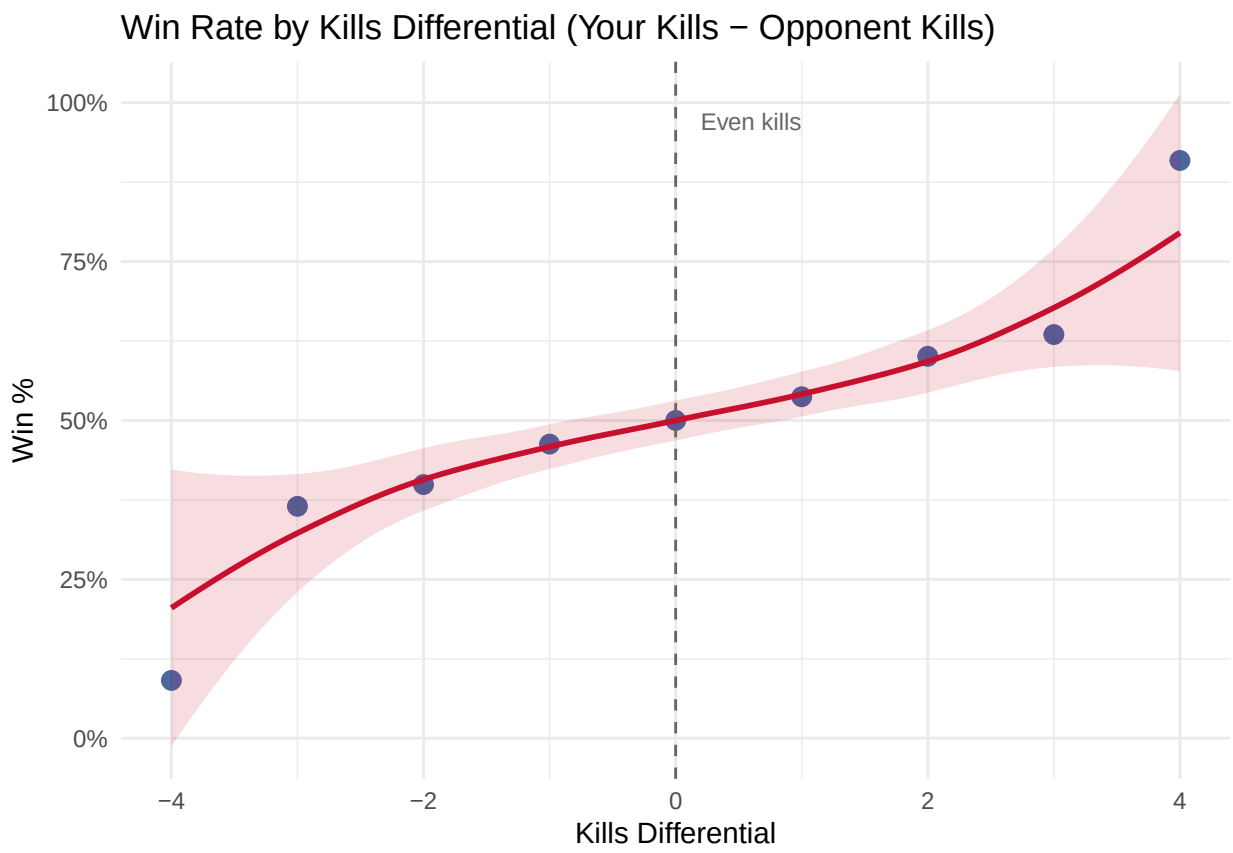
```r
print(p_dist)
```



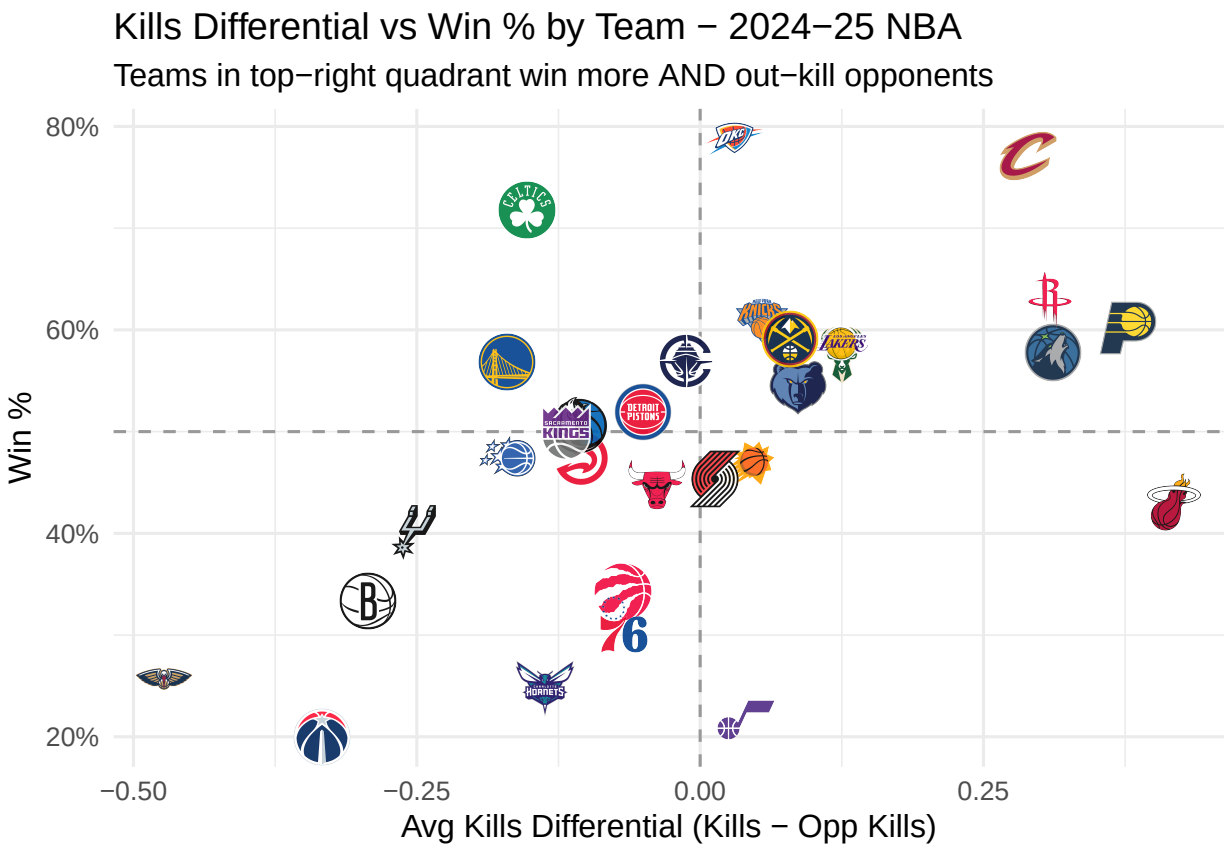

```
print(p_winrate)
```


NBA Win Rate by Kills − Full League 2024−25
60%
55%
50%
45%
Win %
0
2
4
6
Kills
Miller's
threshold

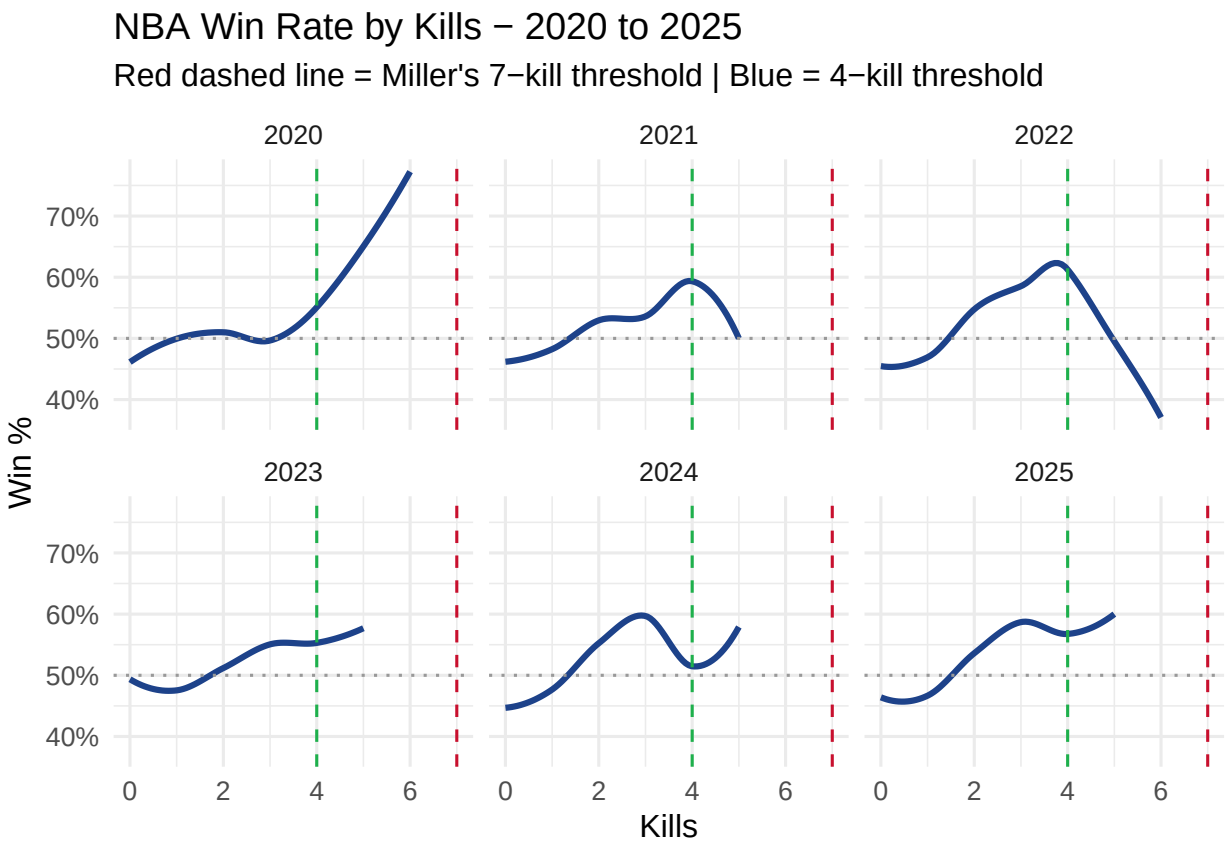
```
print(p_diff)
```


Win Rate by Kills Differential (Your Kills − Opponent Kills)
Even kills
100%
75%
50%
25%
0%
Win %
−4
−2
0
2
4
Kills Differential

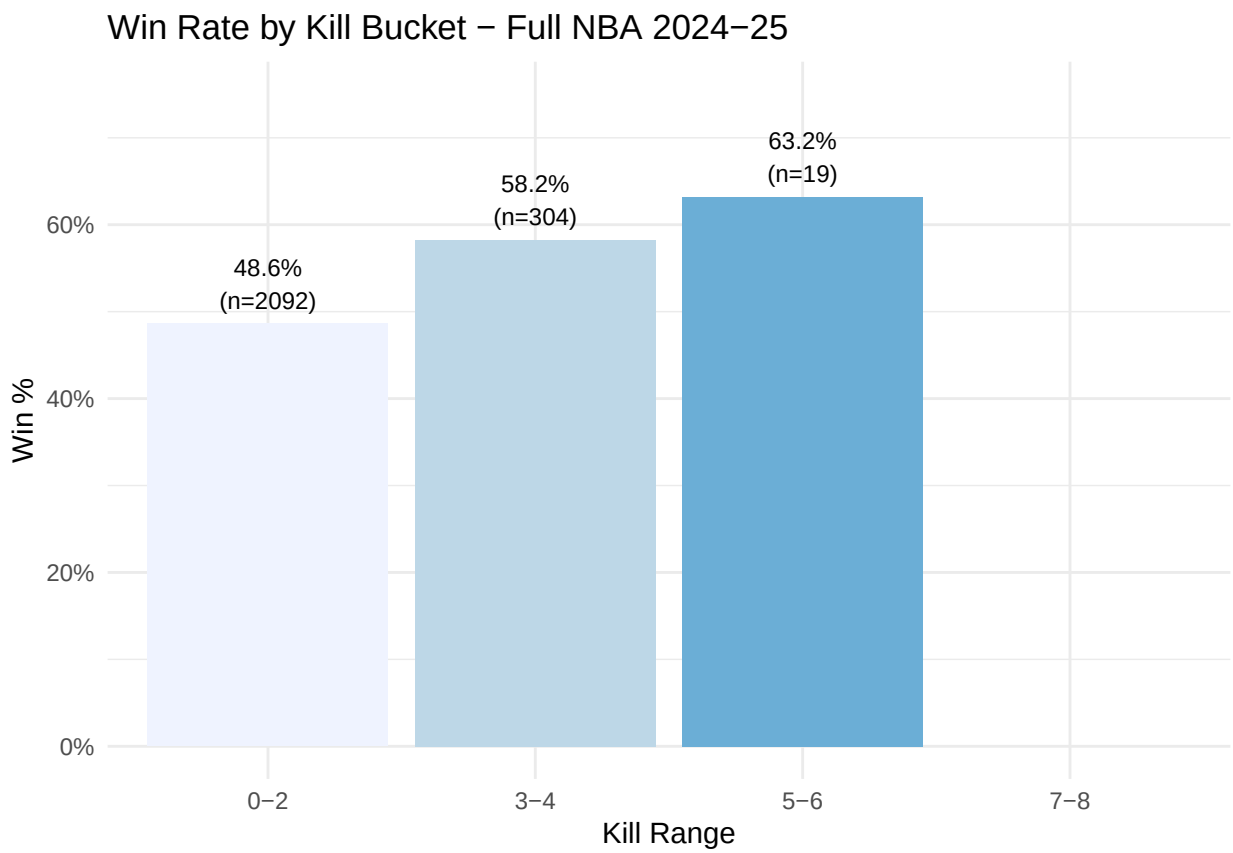
```
print(p_team)
```




```
print(p_multi)
```




```
print(p_bucket)
```



3. Kills Table

```
library(gt)
```

```
## Warning: package 'gt' was built under R version 4.4.3
```

```
library(nbaplotR)

kills_table <- team_kill_avgs %>%
  filter(!team_abbreviation %in% ALL_STAR_TEAMS) %>%
  arrange(desc(avg_diff)) %>%
  mutate(rank = row_number()) %>%
  select(rank, team_abbreviation, avg_kills, avg_opp_kills, avg_diff, win_pct) %>%
  gt() %>%
  nbaplotR::gt_nba_logos(columns = "team_abbreviation") %>%
  fmt_number(columns = c(avg_kills, avg_opp_kills, avg_diff), decimals = 2) %>%
  fmt_percent(columns = win_pct, decimals = 1) %>%
  cols_label(
    rank = "Rank",
    team_abbreviation = "Team",
    avg_kills = "Avg Kills",
    avg_opp_kills = "Avg Opp Kills",
    avg_diff = "Differential",
```

```r
    win_pct = "Win %"
  ) %>%
  data_color(
    columns = avg_diff,
    fn = scales::col_numeric(
      palette = c("#C8102E", "white", "#1D428A"),
      domain = NULL
    )
  ) %>%
  data_color(
    columns = win_pct,
    fn = scales::col_numeric(
      palette = c("white", "#1D428A"),
      domain = c(0, 1)
    )
  ) %>%
  tab_header(
    title = "NBA Kills per Game - 2024-25 Season",
    subtitle = "Sorted by avg kills | Color intensity = magnitude"
  )
```